

계층형 쿼리로 조직도와 사이트맵 만들기

레벨별 조회 + UNION → WITH RECURSIVE 재귀 쿼리 → 메뉴 테이블로 사이트맵 완성

선수학습	JOIN(SELF JOIN), SUBQUERY(IN · WITH), DDL(자기참조 FK), 집합연산자 UNION
이번 챕터	계층형 데이터, 레벨별 UNION 으로 조직도, WITH RECURSIVE 구조와 사용법, 메뉴 테이블 사이트맵
DB 기준	MySQL 8.0 (WITH RECURSIVE 는 8.0부터 지원)
실습 데이터	공통 스키마의 EMP (21명, MANAGER_ID 자기참조) + 이 챕터에서 만드는 MENU

학습 목표

- 한 테이블 안에서 부모-자식이 반복되는 **계층형(트리) 데이터**를 EMP.MANAGER_ID 로 설명할 수 있다.
- **관리자가 없는 직원(1레벨)**을 찾고, 그 사번을 **MANAGER_ID** 로 가진 **직원(2레벨)**을 조회해 UNION 으로 2단계 조직도를 만들 수 있다.
- 위 방식의 한계를 통해 **WITH RECURSIVE** 가 필요한 이유를 설명하고, 앵커 멤버·재귀 멤버·종료 조건을 구분해 쓸 수 있다.
- 재귀 쿼리로 **LEVEL** 과 경로를 계산해 트리 모양의 전체 조직도를 출력할 수 있다.
- 자기참조 **PARENT_ID** 를 가진 **MENU** 테이블을 만들고, 재귀 쿼리로 **사이트맵**을 출력할 수 있다.

1. 계층형 데이터와 조직도

계층형 데이터는 "같은 종류의 행끼리 부모-자식으로 이어져 트리를 이루는" 데이터입니다. 조직도(사원 위에 관리자, 그 위에 또 관리자), 카테고리(대분류 > 중분류 > 소분류), 그리고 이 챕터 마지막에 만들 **화면 메뉴**가 모두 계층형입니다.

EMP 테이블의 **MANAGER_ID** 는 *다른 테이블*이 아니라 **같은 EMP 테이블의 EMP_ID** 를 가리킵니다(자기참조). 즉 "이 사원의 관리자는 누구인가"가 한 테이블 안에서 계속 이어집니다.

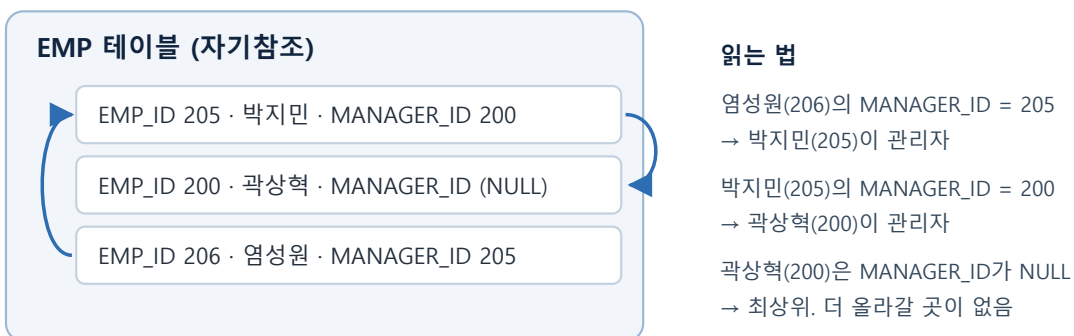


그림 1. **MANAGER_ID** 가 같은 테이블의 **EMP_ID** 를 가리킨다

실제 데이터에서 **MANAGER_ID IS NULL** 인 곽상혁(200)을 뿌리로 하면 아래 트리가 됩니다.

곽상혁 (200, 대표)

└ 권진우 (201) ─ 김민혜 (202)

└ 김은민 (203) ─ 김태일 (204)

└ 박지민 (205) ┐ 염성원 (206) ─ 최주호 (209)

| └ 유제영 (207)

| └ 윤정주 (208)

└ 이광렬 (210) ┐ 이금빈 (211)

| └ 오미자 (212)

└ 이다현 (213) ┐ 전태성 (214)

└ 한재현 (215)

가장 깊은 경로는 **곽상혁 → 박지민 → 염성원 → 최주호**로 4단계입니다. (사번 216~220의 5명은 `MANAGER_ID`가 없어 이 트리에 속하지 않고, 각자 홀로 최상위입니다.)

2. 재귀 없이 조직도 만들기 — 레벨별 조회 + UNION

재귀 문법을 배우기 전에, "레벨을 하나씩 직접 조회해서 이어 붙이는" 방식으로 조직도를 만들어 봅니다. 이 과정을 이해하면 다음 절의 `WITH RECURSIVE` 가 무엇을 자동화하는지 분명해집니다.

2.1 1레벨 — 관리자가 없는 직원

조직의 최상위는 "위에 아무도 없는 사람", 즉 `MANAGER_ID` 가 `NULL` 인 직원입니다.

```
SELECT EMP_ID, EMP_NAME, MANAGER_ID
FROM EMP
WHERE MANAGER_ID IS NULL;
```

출력 결과

EMP_ID	EMP_NAME	MANAGER_ID
200	곽상혁	(null)
216	박홍주	(null)
217	심재호	(null)
218	엄용민	(null)
219	조정원	(null)
220	한규원	(null)

2.2 2레벨 — 그 사번을 `MANAGER_ID` 로 가진 직원

1레벨에서 조회된 사원번호를 `MANAGER_ID` 로 가진 직원이 2레벨(최상위의 직속 부하)입니다. "1레벨 사번 목록"은 서브쿼리로 그대로 가져옵니다.

```
SELECT EMP_ID, EMP_NAME, MANAGER_ID
FROM EMP
WHERE MANAGER_ID IN (
    SELECT EMP_ID FROM EMP WHERE MANAGER_ID IS NULL    -- 1레벨의 사번들
);
```

출력 결과

EMP_ID	EMP_NAME	MANAGER_ID
201	권진우	200
203	김은민	200
205	박지민	200
210	이광렬	200
213	이다현	200

모두 `MANAGER_ID = 200` 입니다. 216~220을 관리자로 둔 사람은 없기 때문에 곽상혁의 부하 5명만 나옵니다.

2.3 UNION으로 두 레벨을 합치기 = 2단계 조직도

두 조회 결과를 UNION ALL로 이어 붙이고, 각 조각에 몇 레벨인지 LVL 값을 상수로 붙입니다.

```
-- 1레벨: 관리자 없는 최상위
SELECT EMP_ID, EMP_NAME, MANAGER_ID, 1 AS LVL
FROM EMP
WHERE MANAGER_ID IS NULL

UNION ALL

-- 2레벨: 1레벨 사번을 MANAGER_ID로 가진 직원
SELECT EMP_ID, EMP_NAME, MANAGER_ID, 2 AS LVL
FROM EMP
WHERE MANAGER_ID IN (
    SELECT EMP_ID FROM EMP WHERE MANAGER_ID IS NULL
)

ORDER BY LVL, MANAGER_ID, EMP_ID;
```

출력 결과

EMP_ID	EMP_NAME	MANAGER_ID	LVL
200	곽상혁	(null)	1
216	박홍주	(null)	1
217	심재호	(null)	1
218	엄용민	(null)	1
219	조정원	(null)	1
220	한규원	(null)	1
201	권진우	200	2
203	김은민	200	2
205	박지민	200	2
210	이광렬	200	2
213	이다현	200	2

1레벨 (LVL 1)

MANAGER_ID IS NULL
→ 곽상혁 외 6명

2레벨 (LVL 2)

MANAGER_ID IN (1레벨 사번)
→ 권진우 ... 이다현 5명

UNION ALL

2단계 조직도 (한 결과로)

LVL 1 곽상혁 · 박홍주 · ...
LVL 2 권진우 · 김은민 · 박지민 ...

3레벨을 보려면? SELECT 조각을
또 붙이고 서브쿼리를 또 중첩...
= 깊이만큼 쿼리가 길어진다

그림 2. 레벨별로 조회한 결과를 UNION ALL로 이어 붙여 조직도를 만든다

2.4 다른 각도 — 집계 함수로 "관리자별 요약" 보기

트리를 위에서 아래로 펼치는 대신, **집계 함수**로 "관리자마다 직속 부하가 몇 명이고 누구인지"를 한 줄씩 요약할 수도 있습니다. `MANAGER_ID` 로 `GROUP BY` 한 뒤 `COUNT` 와 `GROUP_CONCAT` (그룹 내 값을 한 문자열로 이어 붙이는 집계 함수)을 씁니다.

```
SELECT m.EMP_NAME AS 관리자,
       COUNT(*) AS 직속부하수,
       GROUP_CONCAT(e.EMP_NAME ORDER BY e.EMP_ID SEPARATOR ', ') AS 직속부하
FROM EMP e
JOIN EMP m ON e.MANAGER_ID = m.EMP_ID    -- e = 부하, m = 그 관리자
GROUP BY e.MANAGER_ID, m.EMP_NAME
ORDER BY 직속부하수 DESC;
```

출력 결과

관리자	직속부하수	직속부하
곽상혁	5	권진우, 김은민, 박지민, 이광렬, 이다현
박지민	3	염성원, 유제영, 윤정주
이광렬	2	이금빈, 오미자
이다현	2	전태성, 한재현
권진우	1	김민혜
김은민	1	김태일
염성원	1	최주호

집계는 "한 단계 요약", 재귀는 "전체 전개"

`GROUP BY + COUNT / GROUP_CONCAT` 은 **관리자 → 직속 부하 한 단계**를 깔끔히 요약합니다. 하지만 손자·증손자 세대까지 내려가지는 못하고, `GROUP BY` 는 오히려 행을 묶어 계층을 **없애**줍니다. "누가 누구 밑에 몇 단계로 있는지"가 필요하면 다음 절의 재귀 쿼리가 답입니다.

2.5 이 방식들의 한계

- **깊이만큼 조각을 손으로 써야 한다.** 3레벨은 "2레벨 사번을 `MANAGER_ID` 로 가진 직원"을 `UNION ALL` 로 하나 더 붙여야 하고, 그 조건은 서브쿼리를 한 겹 더 중첩해야 합니다. 4레벨은 또 한 겹....
- **조직 깊이를 미리 알아야 한다.** 개편으로 5단계가 생기면 쿼리 자체를 고쳐야 합니다.
- **LVL 을 상수로 박는다.** 조각마다 `1 AS LVL`, `2 AS LVL` ... 을 직접 적습니다.
- **집계 함수(2.4)는 한 단계까지만.** 관리자별 요약은 되지만 전체 트리를 깊이대로 펼치지는 못합니다.

필요한 것은 "몇 단계인지 몰라도 더 붙일 자식이 없을 때까지 알아서 반복해주는" 도구입니다. 그것이 `WITH RECURSIVE` 입니다.

3. WITH RECURSIVE — 소개와 사용법

서브쿼리 챕터의 `WITH` (공통 테이블 식, CTE)는 "이름 붙인 임시 결과"를 만듭니다. 여기에 `RECURSIVE` 를 붙이면 그 CTE가 **자기 자신을 다시 참조**할 수 있어, 결과가 더 늘어나지 않을 때까지 반복됩니다. 2절의 `UNION` 조각들을 하나로 접은 것이라고 보면 됩니다.

3.1 구조

```
WITH RECURSIVE cte_이름 AS (  
  /* (1) 앵커 멤버 : 반복의 시작점. 재귀 없이 딱 1번 실행 */  
  SELECT ... FROM 원본테이블 WHERE 루트조건  
  
  UNION ALL          -- (2) 두 결과를 이어 붙임 (컬럼 개수·타입 일치)  
  
  /* (3) 재귀 멤버 : cte_이름을 다시 참조 → 자식 한 세대를 더 붙임 */  
  SELECT ... FROM 원본테이블 t  
  JOIN cte_이름 c ON t.부모컬럼 = c.식별자  
)  
SELECT * FROM cte_이름; -- (4) 완성된 가상 테이블을 조회
```

3.2 사용법 — 네 부분만 채우면 된다

부분	무엇을 적나	조직도에서는
(1) 앵커 멤버	트리의 시작 행. 한 번만 실행된다.	<code>WHERE MANAGER_ID IS NULL</code> (= 2.1의 쿼리)
(2) <code>UNION ALL</code>	앵커 결과와 재귀 결과를 계속 합친다.	세대가 쌓일 때마다 행 추가
(3) 재귀 멤버	직전 결과(CTE)와 원본을 조인해 "자식 한 세대"를 더 찾는다.	<code>EMP.MANAGER_ID = 직전결과.EMP_ID</code>
(4) 종료	재귀 멤버가 행을 0건 반환하면 반복이 끝난다. (직접 쓰지 않아도 자동)	맨 아래 사원(부하 없음)에 닿으면 멈춤

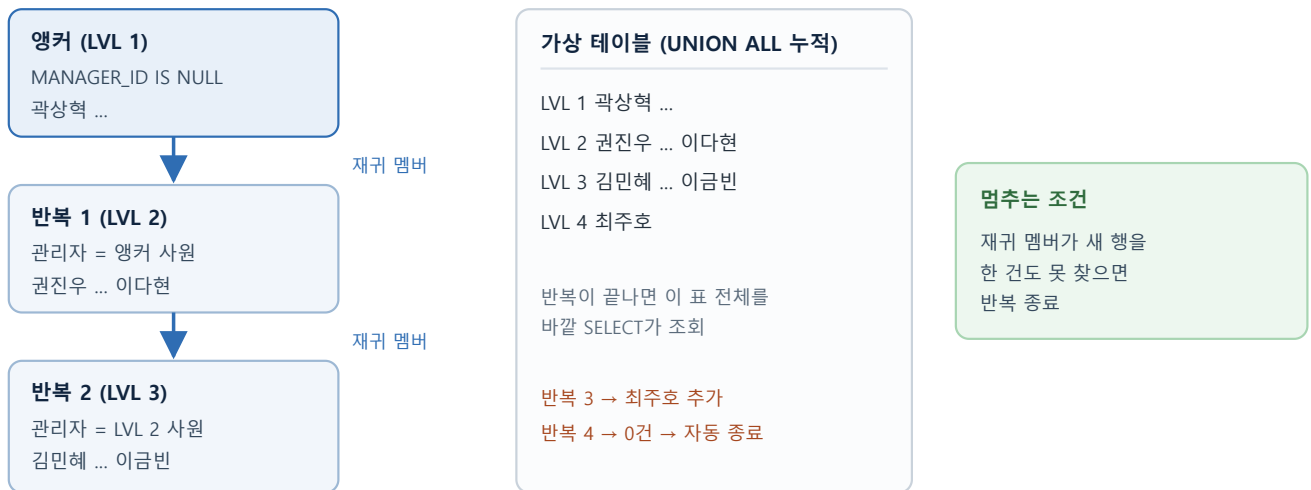


그림 3. 앵커에서 시작해 한 세대씩 자식을 붙여 누적하고, 더 붙일 자식이 없으면 멈춘다

3.3 조직도 완성 쿼리

```
WITH RECURSIVE EmpTree AS (
  -- (1) 앵커: 관리자가 없는 최상위 (2.1의 쿼리와 동일)
  SELECT EMP_ID, EMP_NAME, MANAGER_ID,
         1 AS LVL,
         CAST(EMP_NAME AS CHAR(200)) AS PATH
  FROM EMP
  WHERE MANAGER_ID IS NULL

  UNION ALL

  -- (3) 재귀: 직전 결과(t)의 사원을 관리자로 둔 부하 직원(e)을 붙인다
  SELECT e.EMP_ID, e.EMP_NAME, e.MANAGER_ID,
         t.LVL + 1,
         CONCAT(t.PATH, ' > ', e.EMP_NAME)
  FROM EMP e
  JOIN EmpTree t ON e.MANAGER_ID = t.EMP_ID
)
-- (4) 완성된 가상 테이블을 경로순 정렬 + 깊이만큼 들여쓰기
SELECT EMP_ID,
       CONCAT(REPEAT(' ', LVL - 1), '└─ ', EMP_NAME) AS 조직도,
       MANAGER_ID, LVL, PATH AS 전체경로
FROM EmpTree
ORDER BY PATH;
```

출력 결과 (곽상혁 트리 부분)

EMP_ID	조직도	LVL	전체경로
200	└ 곽상혁	1	곽상혁
201	└ └ 권진우	2	곽상혁 > 권진우
202	└ └ └ 김민혜	3	곽상혁 > 권진우 > 김민혜
203	└ └ 김은민	2	곽상혁 > 김은민
204	└ └ └ 김태일	3	곽상혁 > 김은민 > 김태일
205	└ 박지민	2	곽상혁 > 박지민
206	└ └ 염성원	3	곽상혁 > 박지민 > 염성원
209	└ └ 최주호	4	곽상혁 > 박지민 > 염성원 > 최주호
207	└ └ 유제영	3	곽상혁 > 박지민 > 유제영
208	└ └ 윤정주	3	곽상혁 > 박지민 > 윤정주
...			
216	└ 박홍주	1	박홍주 (관리자·부하가 없는 1인 최상위)

사용 팁

경로 (PATH)로 정렬하면 같은 부모의 자식들이 부모 바로 아래 모여 트리 모양이 됩니다. **특정 부서만 보고 싶**으면 앵커의 WHERE 를 EMP_ID = '205' 처럼 바꾸면 그 아래 부분 조직도만 나옵니다(재귀 멤버는 그대로).

왜 앵커에서 CAST(EMP_NAME AS CHAR(200)) 을 하는가

재귀 CTE는 결과 컬럼의 **자료형을 앵커 멤버 한 곳만 보고 확정**합니다. 재귀 멤버는 실행 중에 만들어지므로 타입 결정에 참여하지 못하고, **이미 정해진 타입에 값이 맞춰 담길** 뿐입니다.

- 앵커가 EMP_NAME (예: VARCHAR(20))을 그대로 보내내면 → PATH 컬럼폭이 **20자로 고정**됩니다.
- 재귀하며 CONCAT(t.PATH, ' > ', e.EMP_NAME) 로 길어진 경로가 20자를 넘으면 → MySQL 8.0.18 이하 ·strict 모드에서는 **ERROR 1406 Data too long**, 그 외에는 **20자에서 조용히 잘려** 경로가 깨집니다.

그래서 앵커에서 미리 **CHAR(200)** 처럼 **깊은 경로까지 담을 폭을 확보**합니다. 일반 UNION 은 모든 SELECT 를 미리 보고 가장 넓은 타입·길이를 병합하므로 이 CAST 가 필요 없습니다. **재귀 CTE만 앵커가 타입을 단독으로 결정**하기 때문에 생기는 차이입니다. 길이 기준은 대략 최대깊이 × (이름 최대길이 + 구분자 3자) — 10단계·이름 20자면 약 230자이므로 넉넉히 200~500을 잡습니다. (5절 메뉴의 SORT_PATH 도 같은 이유로 앵커에서 CAST(... AS CHAR(200)) 을 합니다.)

3.4 UNION 방식과 비교

항목	레벨별 UNION (2절)	WITH RECURSIVE (3절)
깊이	UNION 조각 수만큼 고정	자식이 없을 때까지 자동
단계 추가 시	조각 + 서브쿼리 중첩을 더 작성	수정 불필요
LVL ·경로	상수로 직접 박음	재귀하며 +1, CONCAT 으로 누적
지원 버전	모든 버전	MySQL 8.0+

4. 메뉴 테이블로 사이트맵 완성

계층형 쿼리를 실무에서 가장 자주 쓰는 곳이 **화면 메뉴**입니다. "대메뉴 > 서브메뉴 > 하위메뉴"로 이어지는 구조를, `EMP.MANAGER_ID`와 똑같은 방식(자기참조)으로 `MENU` 테이블 하나에 담고 재귀 쿼리로 사이트맵을 출력합니다.

4.1 자기참조 `PARENT_ID`를 가진 `MENU` 테이블

```
CREATE TABLE MENU (  
  MENU_ID    INT          PRIMARY KEY,  
  MENU_NAME  VARCHAR(50) NOT NULL,  
  PARENT_ID  INT,          -- 상위 메뉴의 MENU_ID (대메뉴는 NULL)  
  SORT_ORDER INT          NOT NULL DEFAULT 0, -- 같은 상위 안에서의 표시 순서  
  URL        VARCHAR(100),  
  -- PARENT_ID는 같은 MENU 테이블의 MENU_ID를 참조 (자기참조 FK)  
  CONSTRAINT FK_MENU_PARENT FOREIGN KEY (PARENT_ID) REFERENCES MENU (MENU_ID)  
);
```

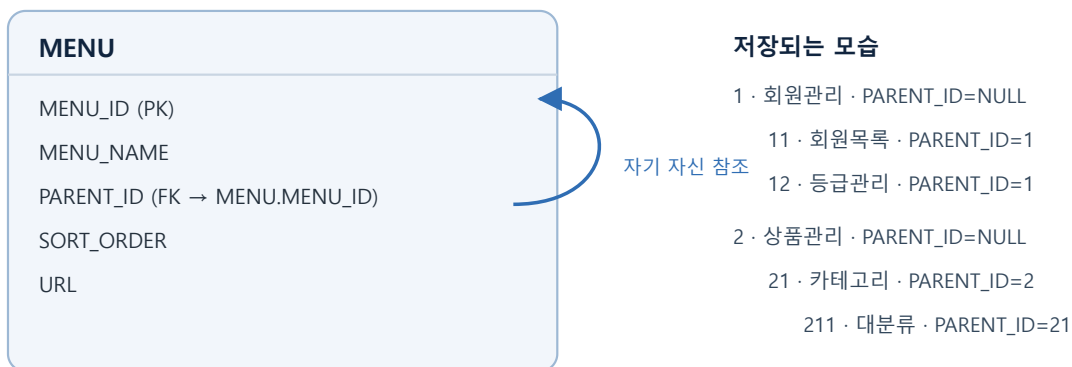


그림 4. `PARENT_ID`가 상위 메뉴의 `MENU_ID`를 가리킨다 — `EMP.MANAGER_ID`와 같은 구조

4.2 데이터 입력 — 상위 메뉴 먼저, 하위가 `PARENT_ID` 지정

입력 순서 주의

자기참조 FK가 걸려 있어 `PARENT_ID`가 가리키는 상위 행이 **먼저 존재**해야 합니다. 대메뉴(`PARENT_ID = NULL`)부터 넣고 → 서브메뉴 → 하위메뉴 순으로 입력합니다.

```

-- 1) 대메뉴 (PARENT_ID = NULL)
INSERT INTO MENU VALUES
(1, '회원관리', NULL, 1, NULL),
(2, '상품관리', NULL, 2, NULL),
(3, '주문관리', NULL, 3, NULL);

-- 2) 서브메뉴 (PARENT_ID = 대메뉴 MENU_ID)
INSERT INTO MENU VALUES
(11, '회원목록', 1, 1, '/admin/members'),
(12, '등급관리', 1, 2, '/admin/members/grade'),
(21, '상품목록', 2, 1, '/admin/products'),
(22, '카테고리', 2, 2, NULL),
(31, '주문목록', 3, 1, '/admin/orders'),
(32, '배송관리', 3, 2, '/admin/orders/ship');

-- 3) 하위메뉴 (PARENT_ID = 서브메뉴 22 '카테고리')
INSERT INTO MENU VALUES
(221, '대분류', 22, 1, '/admin/products/cat/1'),
(222, '중분류', 22, 2, '/admin/products/cat/2'),
(223, '소분류', 22, 3, '/admin/products/cat/3');

```

4.3 재귀 쿼리로 사이트맵 출력

조직도 쿼리와 구조가 같습니다. 다른 점은 정렬 — 같은 상위 메뉴 안에서 `SORT_ORDER` 순서를 지켜야 하므로, 경로를 이름이 아니라 `LPAD(SORT_ORDER, 3, '0')` 를 이어 붙인 **정렬용 경로**로 만듭니다.

```

WITH RECURSIVE SiteMap AS (
  -- (1) 앵커: 대메뉴 (상위가 없는 메뉴)
  SELECT MENU_ID, MENU_NAME, PARENT_ID,
         1 AS DEPTH,
         CAST(LPAD(SORT_ORDER, 3, '0') AS CHAR(200)) AS SORT_PATH
  FROM MENU
  WHERE PARENT_ID IS NULL

  UNION ALL

  -- (3) 재귀: 직전 결과(s)를 상위로 둔 하위 메뉴(m)를 붙인다
  SELECT m.MENU_ID, m.MENU_NAME, m.PARENT_ID,
         s.DEPTH + 1,
         CONCAT(s.SORT_PATH, '-', LPAD(m.SORT_ORDER, 3, '0'))
  FROM MENU m
  JOIN SiteMap s ON m.PARENT_ID = s.MENU_ID
)
SELECT MENU_ID,
       CONCAT(REPEAT(' ', DEPTH - 1), MENU_NAME) AS 사이트맵,
       PARENT_ID, DEPTH
FROM SiteMap
ORDER BY SORT_PATH;

```

출력 결과

MENU_ID	사이트맵	PARENT_ID	DEPTH
1	회원관리	(null)	1
11	회원목록	1	2
12	등급관리	1	2
2	상품관리	(null)	1
21	상품목록	2	2
22	카테고리	2	2
221	대분류	22	3
222	중분류	22	3
223	소분류	22	3
3	주문관리	(null)	1
31	주문목록	3	2
32	배송관리	3	2

4.4 응용 — 특정 대메뉴의 하위만

앵커의 조건만 바꾸면 그 메뉴 아래 사이트맵만 나옵니다(재귀 멤버는 그대로).

-- 앵커에서:

WHERE MENU_ID = 2

-- '상품관리'와 그 아래 전체 (221~223 하위메뉴 포함)

자주 하는 실수

- **RECURSIVE** 키워드 누락 → **WITH** 만 쓰면 CTE가 자기 자신을 참조할 수 없어 "테이블이 없다" 오류.
- **앵커·재귀 멤버의 컬럼 개수/순서/타입 불일치** → **UNION ALL** 오류.
- **경로 컬럼폭 부족** → 재귀 중 경로가 잘리거나 **ERROR 1406**. 앵커가 타입을 단독 결정하므로 앵커에서 **CAST(... AS CHAR(200))** 으로 확보(3.3 경고 박스 참고).
- **재귀 멤버의 조인 방향을 반대로** → 아래로 전개해야 하는데 **t.MANAGER_ID = e.EMP_ID** 로 쓰면 엉뚱한 결과. "직전 결과(부모)에 자식을 붙인다" = **e.MANAGER_ID = t.EMP_ID**.
- **메뉴 데이터를 하위부터 INSERT** → 자기참조 FK 위반. 대메뉴(**PARENT_ID NULL**)부터 넣습니다.
- **사이트맵을 이름 경로로 정렬** → 순서가 뒤섞임. **SORT_ORDER** 를 **LPAD** 로 자리 맞춰 이어 붙인 경로로 정렬해야 합니다.
- **순환 데이터(A→B, B→A)** → 재귀가 끝나지 않음. MySQL은 **cte_max_recursion_depth** (기본 1000) 초과 시 오류로 중단합니다.

핵심 요약

구문 / 개념	역할	비고
자기참조 FK	한 테이블 안에서 부모-자식 연결	EMP.MANAGER_ID , MENU.PARENT_ID
WHERE MANAGER_ID IS NULL	트리의 최상위(루트) 조회	재귀 쿼리의 앵커가 됨
WHERE MANAGER_ID IN (SELECT ...) + UNION ALL	레벨을 하나씩 조회해 이어 붙이기	깊이만큼 조각 필요 · 8.0 미만의 방법
GROUP BY MANAGER_ID + COUNT / GROUP_CONCAT	관리자별 직속 부하 수·명단 요약	한 단계까지만 · 계층을 펼치지 못함
WITH RECURSIVE cte AS (앵커 UNION ALL 재귀)	자기 자신을 참조하는 CTE, 자동 반복	MySQL 8.0+
재귀 멤버 JOIN cte ON e.부모 = cte.식별자	직전 결과에 자식 한 세대를 붙임	0건이면 자동 종료
LVL + 1 / CONCAT(PATH, ...)	깊이·경로를 재귀하며 계산	경로로 정렬 → 트리 모양
앵커 WHERE 교체	특정 부서·특정 메뉴 이하 부분 트리	재귀 멤버는 그대로